

O TAD imageRGB

Universidade de Aveiro

Pedro Martelo, João Paulo Graça

126023, 125733



Capítulo 1

Aspetos Gerais

O presente relatório descreve o trabalho desenvolvido no âmbito da Unidade Curricular de Algoritmos e Estrutura de Dados, visando a consolidação de conhecimentos sobre Tipos Abstratos de Dados (TAD), complexidade algorítmica e manipulação dinâmica de memória.

O foco central do projeto é o desenvolvimento do TAD `imageRGB`, que permite representar e manipular imagens digitais a cores. Uma particularidade importante desta estrutura é a sua representação interna, que utiliza um modelo baseado em LUT (Look-Up Table) – a sua utilização permite que múltiplos píxeis partilhem a mesma definição de cor, poupando, assim, memória em imagens com poucas cores distintas e facilitando a alteração global de uma cor.

O trabalho estruturou-se em duas etapas essenciais:

1. Implementação de funcionalidades básicas (Cópia, Comparação e Rotação) e análise formal e experimental da complexidade da função `ImageIsEqual`.
2. Desenvolvimento e comparação de estratégias de preenchimento de regiões (Region Growing by Flood Filling), contrastando uma abordagem recursiva com versões iterativas baseadas em estruturas lineares (Pilha e Fila).

Capítulo 2

Apresentação

Neste capítulo apresenta-se o estudo da complexidade computacional da função `ImageIsEqual`, que tem como objetivo verificar se duas imagens são idênticas. A função percorre as matrizes de píxeis de ambas as imagens simultaneamente. O número total de píxeis de uma imagem é dado pela expressão: $N = \text{largura} \times \text{altura}$.

2.1 Pior caso

Ocorre quando as duas imagens são exatamente iguais (ou diferem apenas no último píxel). Neste cenário, o algoritmo necessita de iterar por todos os N píxeis que compõem a imagem para confirmar a igualdade. Como se efetuam N comparações e cada comparação é uma operação elementar de tempo constante, o custo computacional é linear, denotada por $O(N)$.

2.2 Melhor caso

Ocorre quando as imagens diferem logo no primeiro píxel (coordenada 0,0). O algoritmo deteta a diferença na primeira iteração e termina imediatamente. O número de operações é constante e independente do tamanho da imagem. Assim, a complexidade algorítmica é constante e denota-se por $\Omega(1)$.

2.3 Análise teórica

Para comprovar a análise teórica, foram realizados testes computacionais, visíveis na tabela abaixo, com imagens de dimensões crescentes, contabilizando o número de comparações efetuadas entre os píxeis

Teste	Dimensão da imagem	Nº total de píxeis	Pior caso	Melhor caso
1	100 x 100	10000	10000	1
2	128 x 128	16384	16384	1
3	256 x 256	65536	65536	1
4	512 x 512	262144	262144	1

Tabela 1: Resultados experimentais da análise de complexidade da função ImageIsEqual.

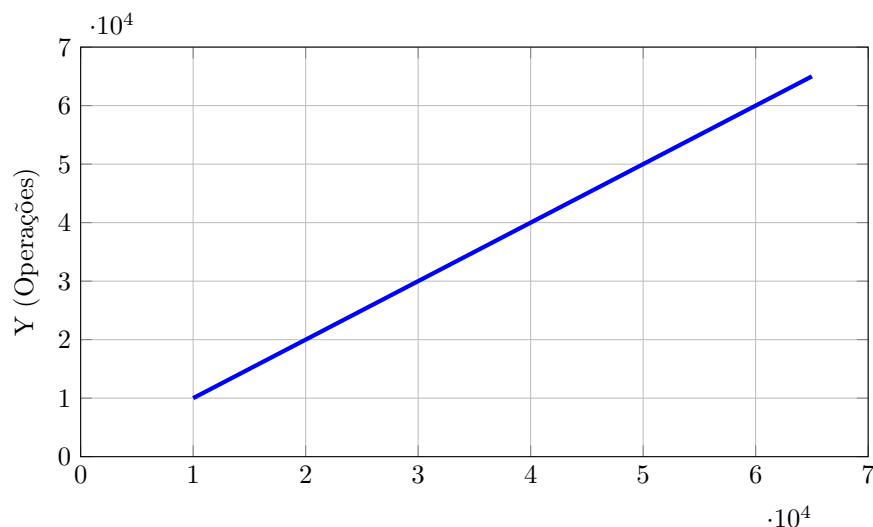


Gráfico 1: Relação linear entre o número de píxeis da imagem e o número de comparações efetuadas pela função ImageIsEqual.

Os dados experimentais apresentados na Tabela 1 corroboram a análise formal. Observa-se que, no pior caso, o número de operações cresce de forma linear em relação ao número total de píxeis (N). Por exemplo, ao aumentar a dimensão da imagem, o número de operações aumenta na mesma proporção, confirmando, assim, a complexidade $O(N)$. Por outro lado, no melhor caso, o número de operações manteve-se constante para todas as dimensões testadas, validando a previsão de complexidade $\Omega(1)$.

2.4 Comparação das estratégias de Region Growing by Flood Filling

2.4.1 Abordagem Recursiva

A função `ImageRegionFillingRecursive` implementa a versão clássica do algoritmo, onde a propagação da cor é realizada através de chamadas sucessivas da própria função para os píxeis vizinhos.

Apresenta uma implementação algorítmica concisa e de lógica direta. Por outro lado, esta estratégia depende da pilha de execução do sistema para armazenar o contexto de cada chamada. Em imagens de grande dimensão, o número de chamadas encadeadas cresce linearmente com a área da região. Isto leva rapidamente ao esgotamento da pilha disponível, provocando um erro de `Stack Overflow`, levando, assim, à falha do programa

2.4.2 Abordagens Iterativas

Para contornar a limitação de memória da recursividade, foram implementadas duas versões iterativas (`ImageRegionFillingWithQUEUE` e `ImageRegionFillingWithSTACK`). Estas funções substituem a pilha do sistema por estruturas de dados explícitas alocadas na memória dinâmica, garantindo robustez para processar imagens de qualquer dimensão.

1. Iterativa com Fila - Ao utilizar uma política First-In, First-Out, o algoritmo processa os píxeis pela ordem de descoberta. O preenchimento expande-se uniformemente em "camadas" concêntricas a partir do ponto inicial.
2. Iterativa com Pilha - Ao utilizar uma política Last-In, First-Out, o algoritmo prioriza o último píxel descoberto. O preenchimento avança até encontrar uma fronteira antes de retroceder.

2.4.3 Síntese da Comparação

A análise efetuada permite concluir que, quanto à robustez, as estratégias iterativas são superiores em cenários reais, pois eliminam o risco de rebentar a pilha do sistema (`Stack Overflow`). Por outro lado, relativamente à eficiência do algoritmo, todas as abordagens são equivalentes ($O(N)$), pois cada píxel da região é visitado e processado um número constante de vezes. Finalmente, abordando o padrão de segmentação, a escolha entre Pilha e Fila não afeta o resultado final da segmentação (a região pintada é a mesma), alterando, apenas, a ordem visual com que os píxeis são percorridos.

Capítulo 3

Conclusão

O trabalho desenvolvido permitiu a aplicação prática e a consolidação de conceitos fundamentais da Unidade Curricular, nomeadamente a construção de Tipos Abstratos de Dados (TAD), a análise de complexidade algorítmica e a gestão de memória dinâmica.

A implementação do TAD imageRGB destacou a importância de uma representação interna eficiente, onde o uso de uma LUT se revelou essencial para a manipulação de cores. Através da análise experimental da função ImageIsEqual, confirmou-se que o custo computacional de processamento de uma imagem tende a crescer linearmente ($O(N)$) com o número de píxeis, validando, assim, as previsões teóricas.

Por fim, o estudo comparativo das estratégias de Region Growing foi determinante para compreender as limitações da recursividade em problemas de grande escala. Demonstrou-se que, para ultrapassar o risco de Stack Overflow, a utilização de abordagens iterativas baseadas em estruturas lineares (Pilha e/ou Fila) é a solução mais adequada. Em suma, todos os objetivos propostos foram atingidos com sucesso, resultando numa solução funcional e eficiente.